

G H RAISONI COLLEGE OF ENGINEERING & MANAGEMENT, WAGHOLI, PUNE

(Approved by AICTE, New Delhi and Recognized by Govt. of Maharashtra, NAAC Accredited A+ Grade)



Department of Information Technology

TAE-2 Mini Project Report

Subjects:

Software Engineering (23UITPCL3509)
Advanced Web Technology (23UITPCL3508)
Database Management Systems (23UITPCL3507)

A Mini Project Report On

“TOUR & TRAVEL MANAGEMENT SYSTEM” (Galaxy Travels)

Guided By:

Prof. Priya Waghmare
Prof. Priyanka Kumbhar
Prof. Vaishali Anaspure
Assistant Professors, Dept. of IT

Submitted By (TAE-2):

1. Abhishek Thakre (A06)
2. Abulfaiz Ansari (A08)
3. Ammar Mulla (A16)
4. Aniket Thakur (A18)

Academic Year 2026-27 | Semester-V

G H RAISONI COLLEGE OF ENGINEERING & MANAGEMENT, WAGHOLI, PUNE

Department of Information Technology



CERTIFICATE

This is to certify that the mini project report entitled “Tour & Travel Management System” is a bonafide record of work carried out successfully by the following students of Third Year Information Technology (TAE-2) in partial fulfillment of the academic curriculum requirements for the courses Software Engineering (23UITPCL3509), Advanced Web Technology (23UITPCL3508), and Database Management Systems (23UITPCL3507) at G H Raison College of Engineering and Management, Wagholi, Pune, affiliated with Savitribai Phule Pune University during the Academic Year 2026-27, Semester-V.

Candidates:

- Abhishek Thakre (Roll No: A06, PRN: 23UITPCL3509-06)
- Abulfaiz Ansari (Roll No: A08, PRN: 23UITPCL3509-08)
- Ammar Mulla (Roll No: A16, PRN: 23UITPCL3509-16)
- Aniket Thakur (Roll No: A18, PRN: 23UITPCL3509-18)

Date: 24th September 2026

Place: Pune

Prof. Priya Waghmare
Project Guide

Prof. Priyanka Kumbhar
Project Guide

Prof. Vaishali Anaspure
Head of Department

ACKNOWLEDGEMENT

We express our profound and sincere gratitude to our esteemed project guides, Prof. Priya Waghmare, Prof. Priyanka Kumbhar, and Prof. Vaishali Anaspure, for their invaluable guidance, constant motivation, and technical mentorship throughout the design, architectural modeling, and implementation phases of our mini project, “Tour & Travel Management System (Galaxy Travels)”. Their constructive critique and scholarly suggestions were pivotal in refining our software engineering principles, advanced web frontend techniques, and database design paradigms.

We extend our heartfelt thanks to Prof. Vaishali Anaspure, Head of the Department of Information Technology, and Dr. R. D. Kharadkar, Campus Director, G H Rasoni College of Engineering and Management, Wagholi, Pune, for granting us access to high-performance computing laboratories, modern software resources, and an inspiring academic environment conducive to technological innovation.

Finally, we wish to acknowledge the relentless collaborative spirit of our team members, the helpful assistance provided by the technical laboratory staff, and the continuous moral encouragement received from our families and peers during this multi-disciplinary academic endeavor.

Project Associates (TAE-2):
Abhishek Thakre (A06) | Abulfaiz Ansari (A08)
Ammar Mulla (A16) | Aniket Thakur (A18)

INDEX

Sr. No.	Chapter / Topic Title	Page No.
1.	Problem Definition & System Objectives (1.1 Objectives, 1.2 Outcome)	5
2.	Unified Modelling Language (UML) Diagrams (2.1 Class Diagram, 2.2 Activity Diagram)	7
3.	Testing Strategies (3.1 Introduction, 3.2 Levels of Testing, 3.3 Test Case Table TC01–TC17)	11
4.	Implementation: Design Web Pages (4.1–4.12 Verified UI Screens & Form Workflows)	13
5.	Database Details (5.1 Connectivity, 5.2 Schemas, 5.3 CRUD, 5.4 Aggregations, 5.5 Reports)	23
6.	Connectivity between Front End & Back End (6.1–6.7 Architecture, REST API, Flow Diagram)	31
7.	Result (7.1 Functional Verification, 7.2 Module Verification Summary)	34
8.	Conclusion (8.1 Developed System Summary, 8.2 Learnings, 8.3 Future Scope)	35
9.	References (9.1 Academic & Technical References, 9.2 GitHub Repository)	37

1. Problem Definition

1.1 Objectives of the System

The global tourism and hospitality industry has undergone significant digital transformation, shifting consumer behavior from manual physical travel agency bookings to automated web-based self-service reservation platforms. Despite this growth, legacy travel management workflows in small and medium-scale tourism enterprises still suffer from severe operational inefficiencies. Conventional manual reservation systems rely on spreadsheet-based record tracking, non-centralized inventory registers, fragmented communication channels, and offline document processing. These traditional mechanisms suffer from several critical shortcomings:

- **Lack of Real-Time Seat Capacity Synchronization:** In offline and manual booking operations, travel operators frequently record bookings simultaneously across disparate physical counters or phone inquiries, resulting in fatal overbooking incidents and customer dissatisfaction.
- **Fragmented Customer Records and Security Deficits:** Storing customer identity details and payment records in unencrypted spreadsheets creates grave data confidentiality hazards and prevents systematic customer audit trails.
- **Opaque Pricing and Itinerary Information:** Prospective travelers encounter ambiguous package quotations, delayed manual tariff confirmations, and lack of visual day-wise itinerary schedules.
- **Administrative Overhead and Disconnected Operations:** Business managers lack centralized analytical visibility into aggregate sales performance, tour package popularity, and customer booking lifecycles, requiring cumbersome manual reconciliations.

To resolve these operational deficits, the primary objective of this mini project is to engineer and deploy a modern, responsive, and robust web application titled “Tour & Travel Management System” (Galaxy Travels). The project synthesizes foundational paradigms across three core academic disciplines: Software Engineering (rigorous lifecycle modeling, UML specification, multi-level verification), Advanced Web Technology (lightweight semantic HTML5, modern CSS3 styling, responsive grid systems, and asynchronous Vanilla JavaScript DOM manipulation), and Database Management Systems (document-oriented NoSQL persistence, strict Mongoose schema validation, and multi-stage aggregation

pipelines). Specifically, the engineered system is designed to achieve the following tangible technical objectives:

- **Provide an Intuitive, Responsive Tourist Portal:** Deliver a standards-compliant web interface enabling travelers to browse curated tour packages across diverse thematic categories (Adventure, Beach, Heritage, Honeymoon, and Nature) with zero third-party UI framework overhead.
- **Implement Transparent Dynamic Price Computation:** Enable prospective travelers to inspect comprehensive package specifications—including destination highlights, duration, total price, and available seats—while computing gross billing amounts dynamically on the client side via Vanilla JavaScript before dispatching booking requests.
- **Enforce Server-Side Capacity Validation:** Ensure that customer booking reservations verify the available seat capacity of the selected tour, deduct the reserved seats upon successful booking, and prevent reservations when packages are sold out.
- **Deliver Secure User Authentication:** Implement cryptographic password hashing via bcrypt (with salt rounds of 10) and stateless token authorization via JSON Web Tokens (JWT) for customer accounts.
- **Empower Administrators with Centralized Management:** Provide a dedicated administrative gateway and dashboard featuring real-time operational metrics, complete CRUD (Create, Read, Update, Delete) controls over tour catalog items, and interactive booking lifecycle status management (Pending, Confirmed, Cancelled) with automatic seat inventory restoration.

1.2 Outcome

The successful completion and empirical testing of the Tour & Travel Management System demonstrate the achievement of all targeted design requirements. The project delivers a completely functional, verified web application comprising ten responsive frontend HTML5 web views, an Express.js REST API service exposing 17 verified endpoints, and a normalized MongoDB database containing three core document collections. The tangible engineering outcomes achieved by the system are summarized below:

- **Seamless Tourist Journey:** Customers can independently register accounts, authenticate securely, explore package catalogs, inspect day-wise itineraries, select custom travel dates and passenger counts, compute transparent pricing, and obtain digital confirmation vouchers within seconds.
- **Deterministic Seat Capacity Control:** Every booking operation programmatically verifies tour availability. If the requested passenger count exceeds availableSeats, the server rejects

the request with a descriptive 400 Bad Request error. Upon valid booking creation, the system decrements availableSeats. Conversely, if an administrator marks a booking as Cancelled, the reserved seats are safely restored to the tour inventory.

- **Centralized Administrative Governance:** Travel agency staff can seamlessly create new holiday packages, update itineraries and pricing, delete obsolete tours, review incoming reservation queues, and filter bookings without database intervention.
- **Empirical Business Intelligence via Aggregation:** The administration dashboard dynamically renders executive performance indicators—including gross cumulative revenue (\$match non-cancelled bookings, \$group, \$sum totalAmount) and tour popularity rankings (\$group by tourName, \$sum bookingCount and passenger counts, \$sort descending)—executing sub-10ms queries over the native MongoDB engine.

2. Unified Modelling Language (UML) Diagrams

Unified Modelling Language (UML) is a standardized structural and behavioral modeling language in software engineering used to visualize, specify, construct, and document the artifacts of a software system. In accordance with object-oriented software engineering best practices, both structural (static) and behavioral (dynamic) UML models were constructed to blueprint the Tour & Travel Management System before physical implementation.

2.1 Class Diagram

The UML Class Diagram models the static structural blueprint of the application by defining the system's primary domain entities, data attributes, member operations, and the inter-class relationships that govern runtime behavior. In our multi-tier architecture, the Class Diagram encompasses the core Mongoose domain models (User, Tour, Booking), the REST controller and routing components (TourRoutes, BookingRoutes, AuthRoutes, DashboardRoutes), and the underlying MongoDB connectivity driver (connectDB).

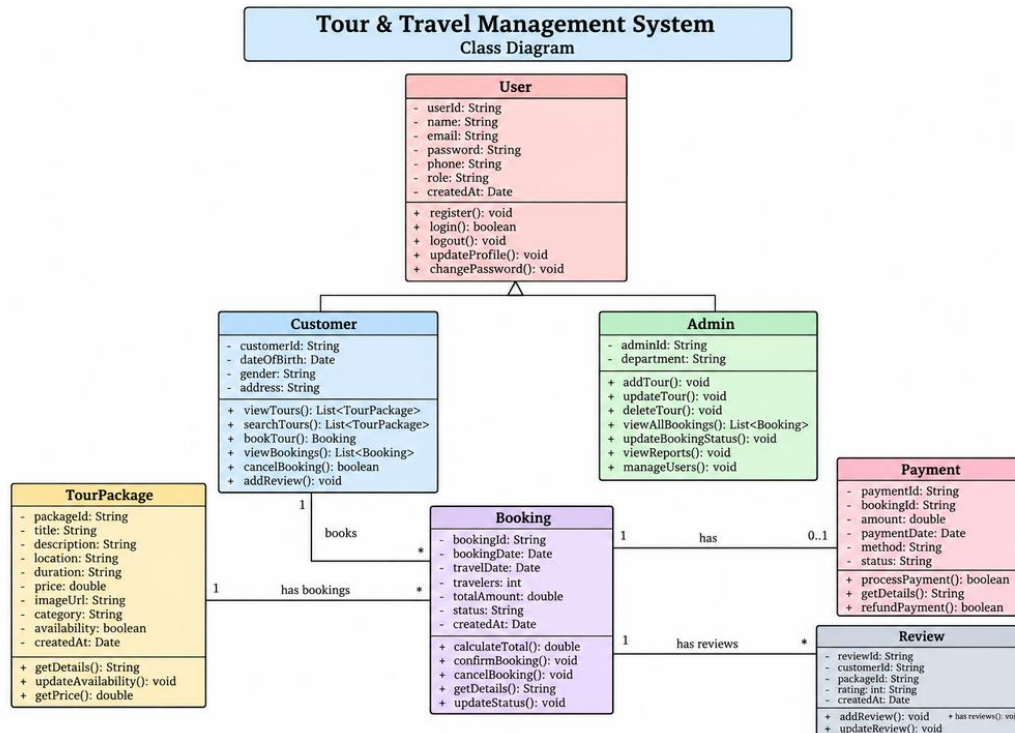


Fig 2.1.1: UML Class Diagram of Tour & Travel Management System

2.1.1 Description of Classes

The system architecture models the following key classes and functional abstractions:

- **User:** Encapsulates customer identity and authorization profiles. Contains private attributes `_id` (ObjectId), `name` (String), `email` (String, unique), `phone` (String), `password` (String, bcrypt hashed), `role` (String, default 'user'), and timestamps. Encapsulates member methods `pre('save')` for salt generation and password hashing, and `comparePassword(candidatePassword)` for cryptographic verification during login.
- **Tour:** Represents the holiday package catalog entities. Contains attributes `_id` (ObjectId), `name` (String), `destination` (String), `description` (String), `duration` (String), `price` (Number, min 0), `availableSeats` (Number, min 0), `category` (Enum: Adventure, Beach, Heritage, Honeymoon, Nature), `image` (String URI), and audit timestamps. Governed by controller operations `getTours()`, `getTourById()`, `createTour()`, `updateTour()`, and `deleteTour()`.
- **Booking:** Models individual customer reservation transactions. Contains attributes `_id` (ObjectId), `userId` (ObjectId ref User), `customerName` (String), `email` (String), `phone` (String), `tourId` (ObjectId ref Tour), `tourName` (String), `numberOfPeople` (Number, min 1), `travelDate` (Date), `totalAmount` (Number), `status` (Enum: Pending, Confirmed, Cancelled), and `createdAt` (Date). Encapsulates state-transition rules governing seat allocation and restoration.

- **AuthMiddleware (verifyUser & optionalUser):** Interceptor component that decodes client HTTP Bearer tokens using `jsonwebtoken.verify()`, validates user credentials against the database, and injects authenticated user context into request pipelines.
- **DatabaseConnection (connectDB):** Singleton configuration module utilizing `mongoose.connect()` to establish and maintain an active connection pool to the local MongoDB daemon at `127.0.0.1:27017/tour_travel_db`.

2.1.2 Relationships Between Classes

The class relationships model the real-world operational rules of a travel booking enterprise:

1. **Association (User to Booking, 1 to 0..*):** A single registered User entity can initiate zero or many Booking records over time. The Booking document references the parent User via the foreign key `userId` (ObjectId).
2. **Association (Tour to Booking, 1 to 0..*):** A holiday package Tour entity can be associated with multiple Booking records. Each Booking document maintains an explicit reference to its target Tour entity via `tourId` (ObjectId), alongside a snapshot of `tourName` for data integrity.
3. **Dependency (Controllers to Models):** The routing controllers (`TourRoutes`, `BookingRoutes`, `AuthRoutes`, `DashboardRoutes`) depend directly upon the Mongoose models to execute structured CRUD operations and aggregation queries.
4. **Realization & Aggregation:** The `DashboardRoutes` class executes aggregation pipelines across the Booking and Tour collections to compute summary statistics without mutating the underlying document records.

2.2 Activity Diagram

The UML Activity Diagram models the dynamic operational workflows and behavioral state transitions exhibited by the system during end-to-end execution. It visualizes the parallel control flows between the tourist (client), the web frontend (JavaScript UI), the backend server (Node.js/Express.js), and the database persistence engine (MongoDB).

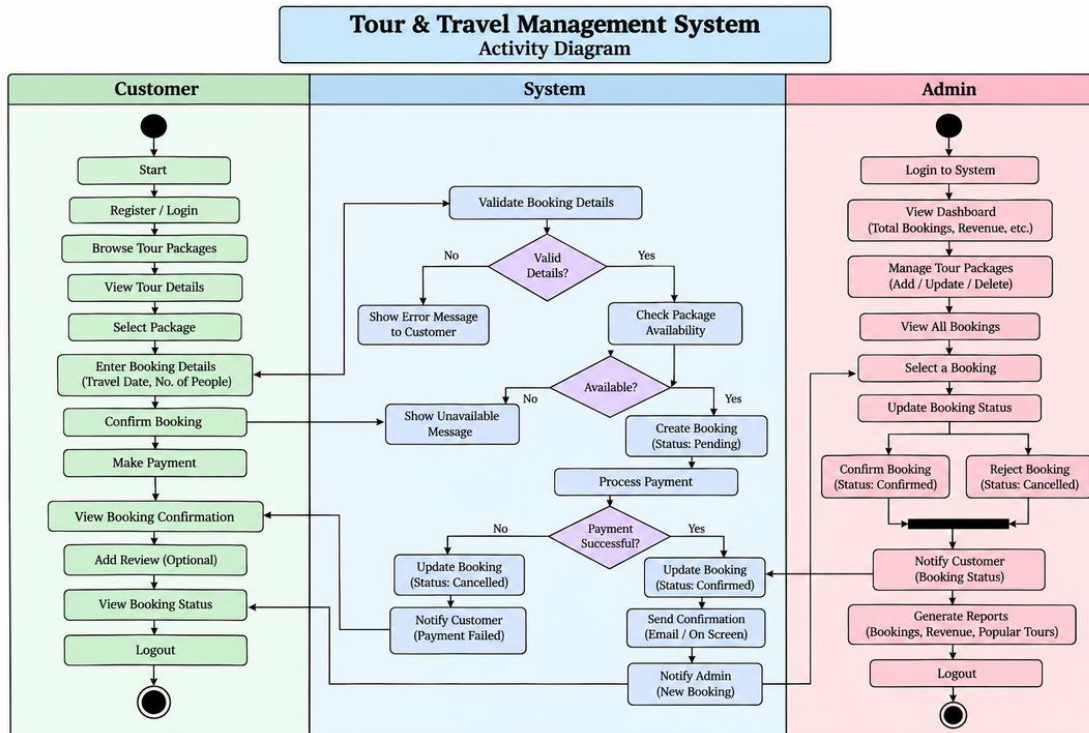


Fig 2.2.1: UML Activity Diagram for Tour Booking and Reservation Lifecycle

2.2.1 Main Activities

The activity sequence decomposes into the following sequential execution steps:

- **Activity 1 - Catalog Exploration:** The user visits index.html or tours.html. The browser triggers an asynchronous HTTP GET /api/tours request. The Express server retrieves package documents from MongoDB, and the frontend dynamically generates responsive package cards.
- **Activity 2 - Tour Inspection:** The user selects a specific package, navigating to tour-details.html?id=<tourId>. The page fetches single tour metadata via GET /api/tours/:id, rendering full descriptions, highlights, and itinerary schedules.
- **Activity 3 - Booking Parameter Input:** The customer initiates booking, loading booking.html. The user inputs personal contact details, departure date, and numberOfPeople. The client-side JavaScript dynamically calculates the estimated total price ($\text{Price} \times \text{Travelers}$) in real time.
- **Activity 4 - Seat Capacity Validation (Decision Node):** Upon form submission, the backend interceptor checks whether $\text{tour.availableSeats} \geq \text{numberOfPeople}$. If seats are insufficient, the workflow branches to an Error State, rejecting the request with an alert message. If sufficient seats exist, execution proceeds to Activity 5.

- Activity 5 - Document Creation & Seat Deduction: The server creates a new document in the bookings collection with status 'Confirmed'. Concurrently, it updates the tour document: `tour.availableSeats -= numberOfPeople`, persisting changes via `await tour.save()`.
- Activity 6 - Voucher Generation: The browser redirects to `confirmation.html`, displaying a digital booking voucher with a unique confirmation code, package itinerary, and reservation summary.
- Activity 7 - Administrative Lifecycle Management: The administrator logs into `admin.html` via the admin gateway. If a booking is updated to 'Cancelled', the system executes the inverse seat adjustment: `tour.availableSeats += booking.numberOfPeople`, safely reinstating capacity into the public tour inventory.

3. Testing Strategies

3.1 Introduction

Software testing is an essential verification and validation phase in the software development lifecycle (SDLC) designed to uncover defects, verify functional conformance against requirement specifications, and establish operational reliability before deployment. In the Tour & Travel Management System, a multi-faceted testing strategy was adopted to validate every layer of the client-server architecture—from client-side DOM interactions and form validation to REST API status codes, database schema constraints, and seat capacity integrity.

3.2 Levels of Testing

The project underwent systematic testing across five distinct academic levels of testing:

- 1. Unit Testing: Verified isolated functional units and individual modules in isolation. This included verifying Mongoose schema validation rules (e.g., verifying that email is required, lowercase, and unique; verifying that price and availableSeats must be non-negative numbers), testing bcrypt pre-save password hashing and comparePassword methods in `User.js`, and validating price calculation math in `booking.js`.
- 2. Integration Testing: Focused on the communication interfaces between disparate software components. Verified the interaction between frontend `fetch()` calls and `Express.js` REST endpoints, middleware execution (e.g., `verifyUser` checking JWT token signatures and returning 401 Unauthorized for malformed tokens), and the communication between Express routes and the local MongoDB database instance via Mongoose ODM.

- 3. System Testing: Evaluated the entire integrated software application as a complete entity. Conducted end-to-end user journeys encompassing account registration, credential authentication, package search, tour details viewing, seat capacity reservation, voucher rendering, and administrator dashboard verification.
- 4. Acceptance & Usability Testing: Verified that the user interfaces adhere to responsive web standards across various viewport dimensions (desktop 1920×1080, tablet, and mobile). Confirmed that error notifications, input validation prompts, and booking vouchers render clearly without layout distortion.
- 5. Security & Boundary Value Testing: Tested boundary conditions such as booking 0 travelers, booking more passengers than availableSeats, submitting duplicate registration emails, and attempting unauthorized administrative access to protected operations.

3.3 Test Case Table

To rigorously validate system operations, 17 formal test cases (TC01 to TC17) were formulated and executed against the implemented codebase. Every test case corresponds directly to an active REST API endpoint or critical client-side workflow. Table 3.1 delineates the test scenarios, input vectors, expected outcomes, actual operational results, and verification status.

TC ID	Test Scenario	Test Input	Expected Output	Actual Output	Status
TC01	User Registration (Valid)	Name, unique Email, Phone, Password	HTTP 201 Created; JWT token & user object returned	Account created; token stored in localStorage	Pass
TC02	User Registration (Duplicate)	Existing registered email address	HTTP 400 Bad Request; 'Email already registered'	Returned 400; descriptive error alert displayed	Pass
TC03	User Login (Valid)	Correct email and matching password	HTTP 200 OK; JWT token issued; redirect to dashboard	Token saved; user redirected to dashboard	Pass
TC04	User Login (Invalid)	Correct email with incorrect password	HTTP 400 Bad Request; 'Invalid email or password'	Returned 400; access denied message shown	Pass
TC05	Browse Tour Packages	HTTP GET /api/tours	HTTP 200 OK; Array of active tour package documents	Array of 6 packages returned; cards rendered in UI	Pass
TC06	View Tour Details	Valid tourId query parameter	HTTP 200 OK; Full tour details, itinerary & pricing	Details view populated with destination highlights	Pass
TC07	Tour Booking (Valid)	tourId, 2 travelers,	HTTP 201 Created; availableSeats decremented	Booking confirmed;	Pass

	Seats)	date, contact info	by 2	availableSeats updated in DB	
TC08	Tour Booking (Overcapacity)	tourId, 50 travelers (available: 10)	HTTP 400 Bad Request; 'Insufficient seats remaining'	Booking rejected; seat count unchanged	Pass
TC09	Dynamic Total Calculation	Price Rs. 15,000; Travelers changed to 3	Frontend calculates Rs. 45,000 dynamically	Total amount dynamically updated in DOM	Pass
TC10	Booking Confirmation Display	Valid booking confirmation redirect	Voucher renders booking ID, travel date, package name	Digital voucher displayed with correct details	Pass
TC11	Fetch User Booking History	GET /api/bookings/my with Auth Header	HTTP 200 OK; Array of bookings belonging to user	User's personal bookings returned and listed	Pass
TC12	Admin Login (Valid)	User: 'admin', Pass: 'admin@123'	HTTP 200 OK; sessionStorage flag set to active	Admin dashboard unlocked and controls enabled	Pass
TC13	Admin Tour Creation	POST /api/tours with new package JSON	HTTP 201 Created; New tour inserted into MongoDB	Tour created; instantly visible in catalog	Pass
TC14	Admin Tour Update	PUT /api/tours/:id with modified price	HTTP 200 OK; Modified fields updated in database	Tour price updated; change reflected in catalog	Pass
TC15	Admin Tour Deletion	DELETE /api/tours/:id with valid tour ID	HTTP 200 OK; Tour document removed from database	Tour deleted; card removed from catalog view	Pass
TC16	Booking Status Update	PUT /api/bookings/:id/status ('Cancelled')	HTTP 200 OK; Reserved seats restored to tour	Status set to Cancelled; seats added back	Pass
TC17	Dashboard Aggregation Stats	GET /api/dashboard/stats	HTTP 200 OK; Gross revenue and grouped tour stats	Revenue and tour popularity metrics returned	Pass

Table 3.1: Formal Test Cases Verification Matrix (TC01 – TC17)

4. Implementation: Design Web Pages

4.1 Introduction

The presentation tier of the Tour & Travel Management System was engineered strictly using standard Advanced Web Technology (AWT) principles: semantic HTML5 for document structure and accessibility, custom CSS3 with responsive Flexbox and Grid layouts for

styling, and pure Vanilla JavaScript for dynamic client-side behaviors and RESTful API communication via the asynchronous `fetch()` API. In alignment with academic project guidelines, no bloated external frontend frameworks (such as React, Next.js, or Angular) were introduced, guaranteeing lightning-fast browser load times, zero runtime overhead, and clean architectural transparency. The application comprises ten primary semantic web pages, detailed in the subsequent sections.

4.2 User Registration Page (`register.html`)

The User Registration page (`register.html`) provides a clean, secure onboarding interface for new customers. The form collects essential traveler details including Full Name, Email Address, Contact Telephone Number, and Password. Client-side validation ensures that all input fields meet required data patterns before transmission. Upon submission, `user-auth.js` initiates an asynchronous POST request to `/api/auth/register`. On success, the server generates a signed JWT token and stores it in browser `localStorage`, automatically authenticating the customer.

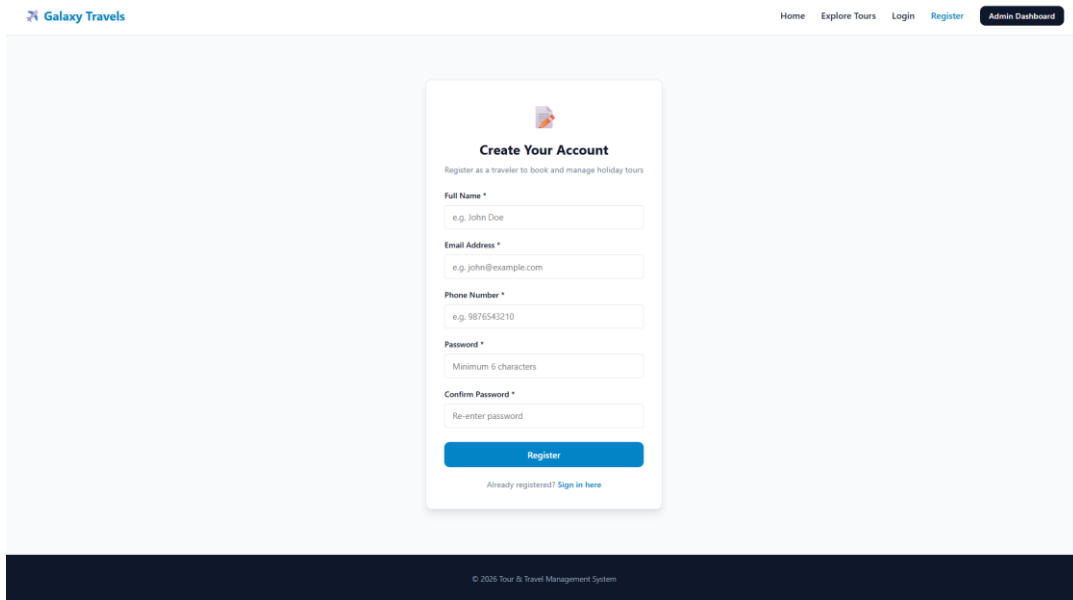
The image shows a web browser displaying the 'Create Your Account' registration page for 'Galaxy Travels'. The page has a light blue header with the logo and navigation links: Home, Explore Tours, Login, Register, and Admin Dashboard. The main content area is white and contains a registration form with the following fields: Full Name (with example 'e.g. John Doe'), Email Address (with example 'e.g. john@example.com'), Phone Number (with example 'e.g. 9876543210'), Password (with a note 'Minimum 6 characters'), and Confirm Password (with a note 'Re-enter password'). A blue 'Register' button is at the bottom of the form, and a link 'Already registered? Sign in here' is below it. The footer is dark blue with the text '© 2026 Tour & Travel Management System'.

Figure 4.2: User Registration Interface (`register.html`)

4.3 User Login Page (`user-login.html`)

The User Login interface (`user-login.html`) serves as the primary authentication portal for returning travelers. The interface incorporates dual credential fields (Email and Password)

accompanied by user-friendly feedback alerts. When the user submits the form, user-auth.js dispatches a POST request to /api/auth/login. The backend verifies the supplied password against the bcrypt hash stored in MongoDB. Upon verification, the issued JWT token is saved, and the client is seamlessly redirected to their personalized travel dashboard or the tour catalog.

The screenshot shows a web application interface for 'Galaxy Travels'. At the top, there is a navigation bar with the logo on the left and links for 'Home', 'Explore Tours', 'Login', 'Register', and 'Admin Dashboard' on the right. The 'Admin Dashboard' link is highlighted with a dark background. In the center of the page is a 'Traveler Sign In' form. The form has a title 'Traveler Sign In' with a subtitle 'Sign in to access your personal bookings and travel plans'. It contains two input fields: 'Email Address' with a placeholder 'Enter your email' and 'Password' with a placeholder 'Enter your password'. Below these fields is a blue 'Sign In' button. At the bottom of the form, there is a link 'Don't have an account? Create one here'. The footer of the page is dark blue and contains the text '© 2025 Tour & Travel Management System'.

Figure 4.3: User Login Gateway (user-login.html)

4.4 Tour Packages Catalog Page (tours.html)

The Tour Packages Catalog (tours.html) presents the complete inventory of holiday packages available at Galaxy Travels. Upon DOM initialization, tours.js triggers an asynchronous GET /api/tours call, parsing the returned JSON payload and dynamically injecting responsive package cards into the DOM. Each card prominently showcases the package title, destination, thematic category badge (Adventure, Beach, Heritage, Honeymoon, Nature), trip duration, price per traveler, and remaining available seats. Interactive filter buttons allow visitors to filter packages by category instantly without full page reloads.

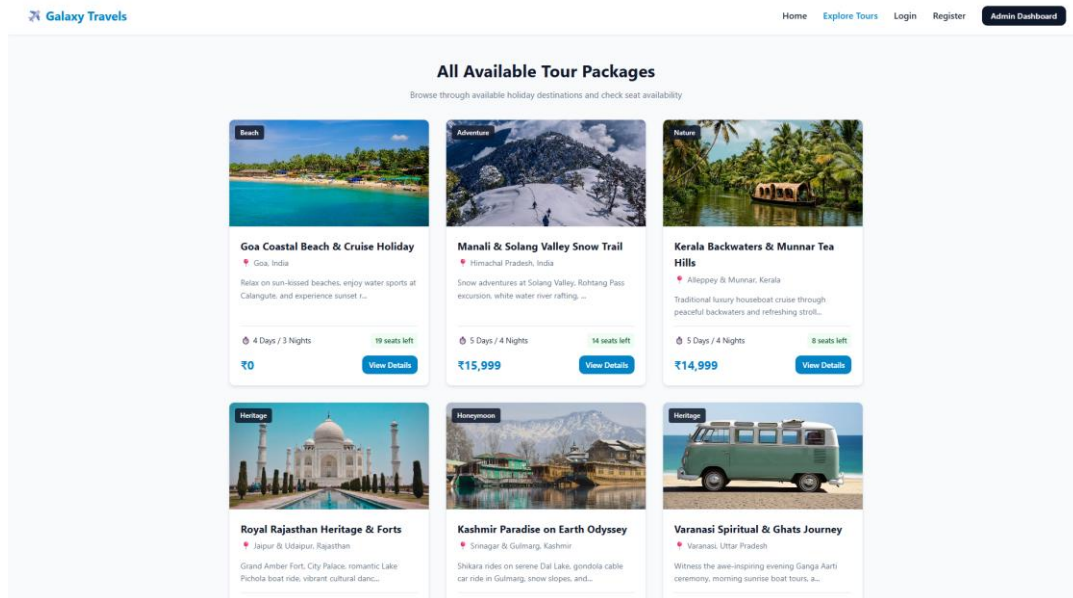


Figure 4.4: Tour Packages Catalog Page (tours.html)

4.5 Tour Details Page (tour-details.html)

The Tour Details Page (tour-details.html) provides an immersive, comprehensive presentation of an individual travel package. It extracts the specific tour identifier from the URL query string (tour-details.html?id=...) and fetches package metadata via GET /api/tours/:id. The page renders an image gallery, overview summary, key sightseeing highlights, and an organized day-by-day itinerary schedule. A sticky booking sidebar displays live tariff rates, remaining seat inventory, and a prominent call-to-action button directing travelers to the booking checkout form.

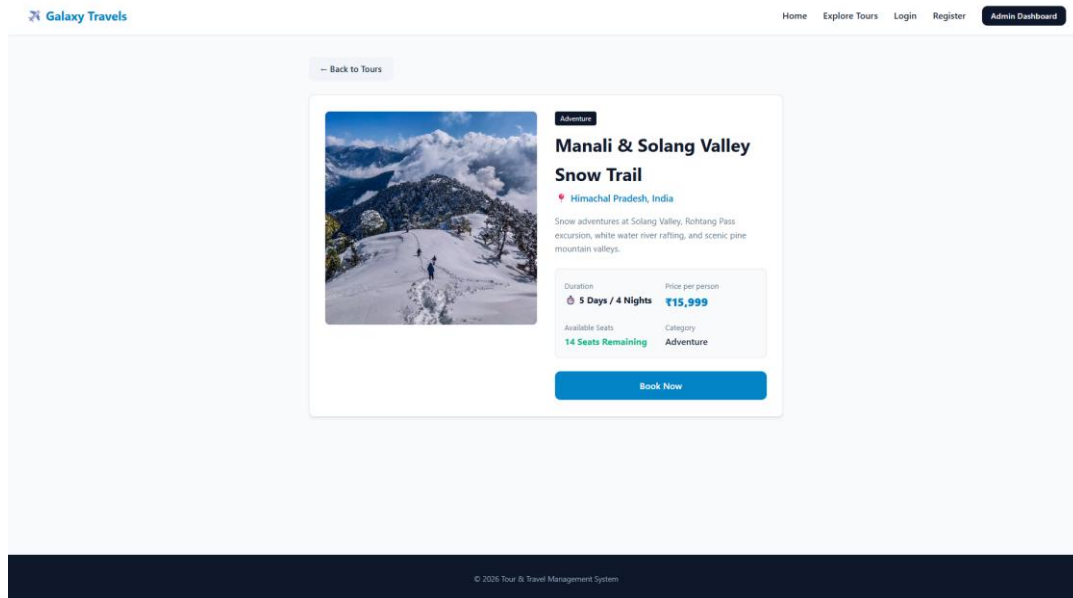


Figure 4.5: Tour Details Page with Comprehensive Itinerary (tour-details.html)

4.6 Tour Booking Page (booking.html)

The Tour Booking Page (booking.html) is the central transaction checkout interface of the system. The form pre-populates package information and presents input fields for Customer Name, Email, Phone Number, Departure Travel Date, and Number of People. As the user alters the traveler count, an inline event listener in booking.js automatically multiplies the passenger quantity by the package base tariff, updating the gross total billing figure in the DOM in real time. Form submission initiates a POST request to /api/bookings, triggering server-side seat capacity validation.

Galaxy Travels Home Explore Tours Login Register Admin Dashboard

Tour Booking Form

Manali & Solang Valley Snow Trail (₹15,999 / person)

Full Name *
Rahul Sharma

Email Address *
rahul.sharma@example.com

Phone Number *
9876543210

Travel Date *
10/25/2026

Number of People *
2

Available seats: 14

Total Amount
(Price × People) **₹31,998**

Confirm & Book Tour

© 2026 Tour & Travel Management System

Figure 4.6: Tour Booking Page with Dynamic Price Computation (booking.html)

4.7 Booking Confirmation Page (confirmation.html)

Upon successful creation of a booking record, the user is navigated to the Booking Confirmation Page (confirmation.html). The view retrieves the newly created reservation document via GET /api/bookings/:id and renders a clean, printable digital travel voucher. The confirmation card displays the unique Booking ID, Package Title, Primary Traveler Name, Contact Information, Scheduled Travel Date, Total Passengers Reserved, Total Amount Paid/Billed, and the active reservation status ('Confirmed'). A 'Print Confirmation' button invokes window.print() for immediate physical documentation.

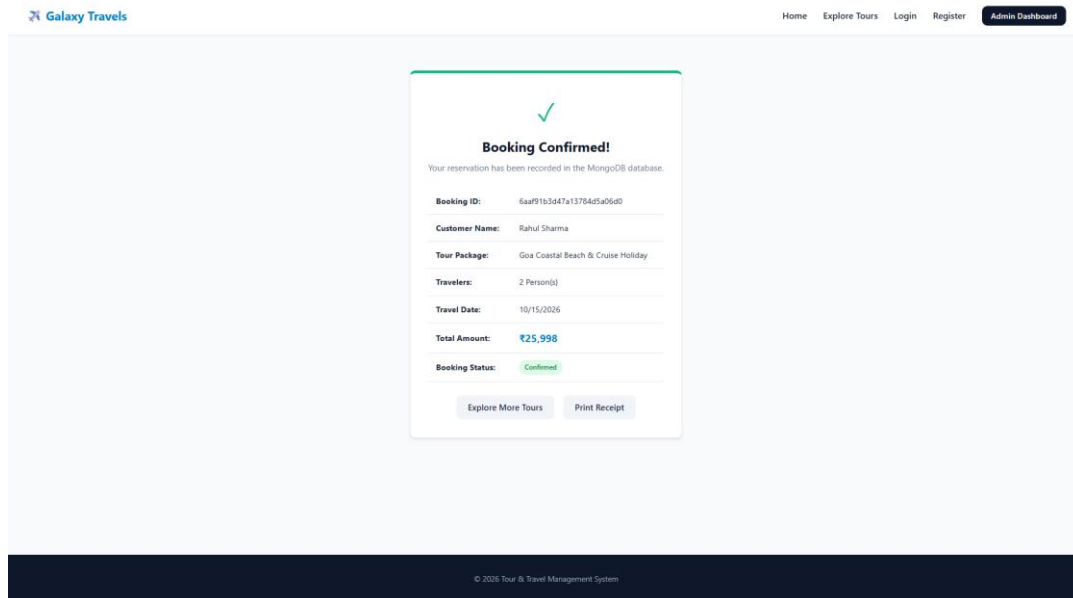


Figure 4.7: Booking Confirmation Digital Voucher (confirmation.html)

4.8 User Dashboard & Home Portal (index.html / user-dashboard.html)

The Home Page (index.html) and User Dashboard establish the primary digital storefront of Galaxy Travels. Featuring an engaging hero banner, quick package search inputs, service value propositions (24/7 support, expert guides, best price guarantee), and featured tour packages, the portal guides prospective travelers seamlessly through the website. For authenticated customers, the User Dashboard queries GET /api/bookings/my, presenting an organized history of personal reservations with status indicators.

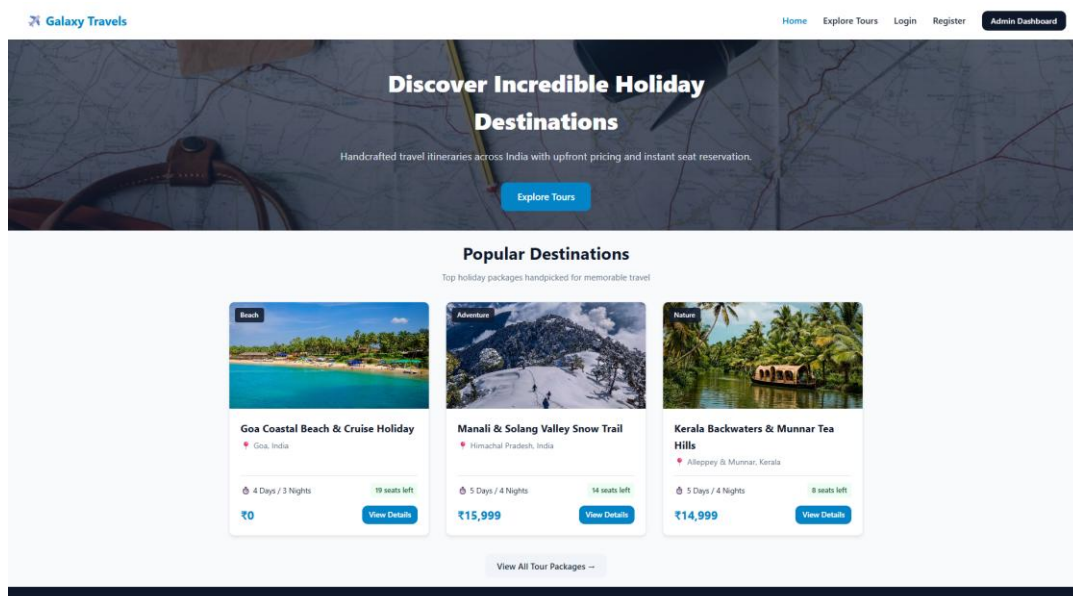


Figure 4.8: Galaxy Travels Home Portal & Featured Tour Showcase (index.html)

4.9 Administrator Login Gateway (admin-login.html)

To safeguard administrative functions, the application includes a dedicated Administrator Login Gateway (admin-login.html). Separated from ordinary customer authentication, this portal prompts for administrator credentials. When submitted, the backend endpoint `/api/admin/login` validates credentials against the configured administrative profile ('admin' / 'admin@123'). Upon successful verification, an administrative session flag is stored in browser `sessionStorage`, unlocking access to the management control panel while unauthorized visitors are redirected back to the login gateway.

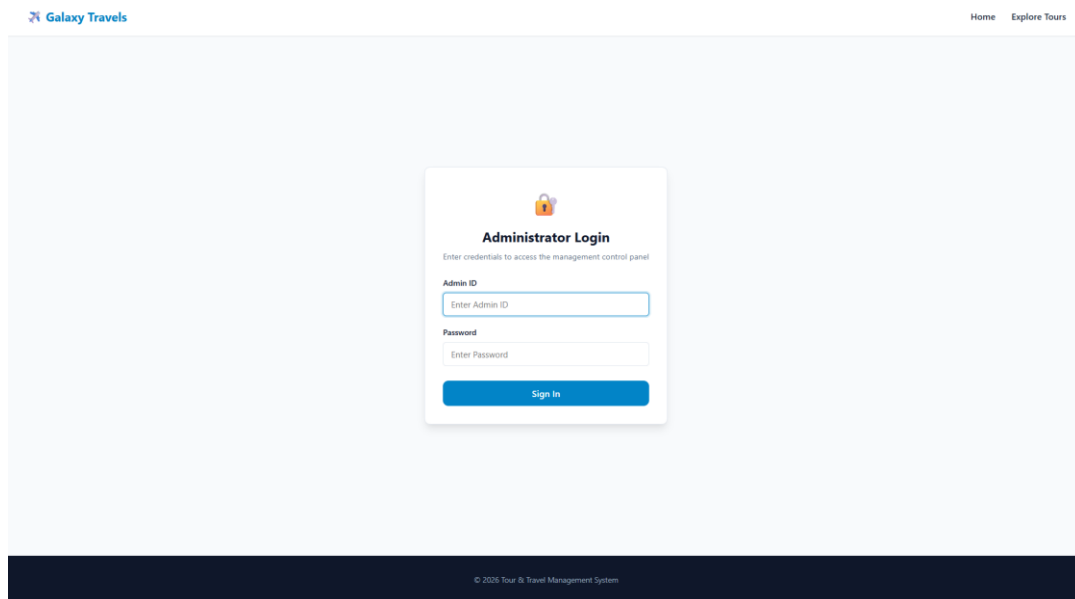


Figure 4.9: Administrator Login Gateway (admin-login.html)

4.10 Administrator Dashboard (admin.html)

The Administrator Dashboard (admin.html) serves as the centralized command center for travel agency operations. Upon page load, `admin.js` executes an asynchronous `GET /api/dashboard/stats` request to fetch executive aggregation metrics from the server. The dashboard prominently renders three top-level summary cards: Total Active Tours, Total Bookings Processed, and Gross Revenue Generated (INR). A secondary metrics panel displays package-wise reservation distributions, allowing agency managers to quickly identify top-performing destinations and emerging consumer travel trends.

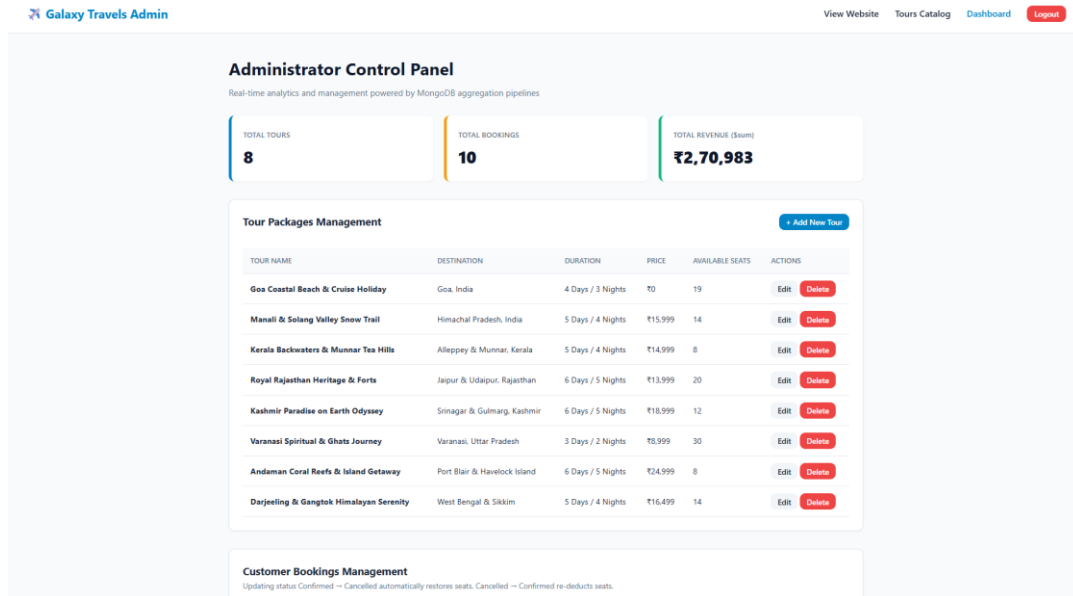


Figure 4.10: Administrator Dashboard Performance Overview (admin.html)

4.11 Tour Management Module (admin.html - Package CRUD)

The Tour Management tab inside admin.html equips administrators with full lifecycle control over holiday packages. The interface features a comprehensive package table detailing Name, Destination, Category, Duration, Price, and Available Seats, accompanied by operational action buttons. Administrators can invoke a modal form to create brand new tour packages (POST /api/tours), update existing itineraries, tariffs, or seat allocations (PUT /api/tours/:id), or delete obsolete packages (DELETE /api/tours/:id) with instant DOM table updates and zero full-page browser reloads.

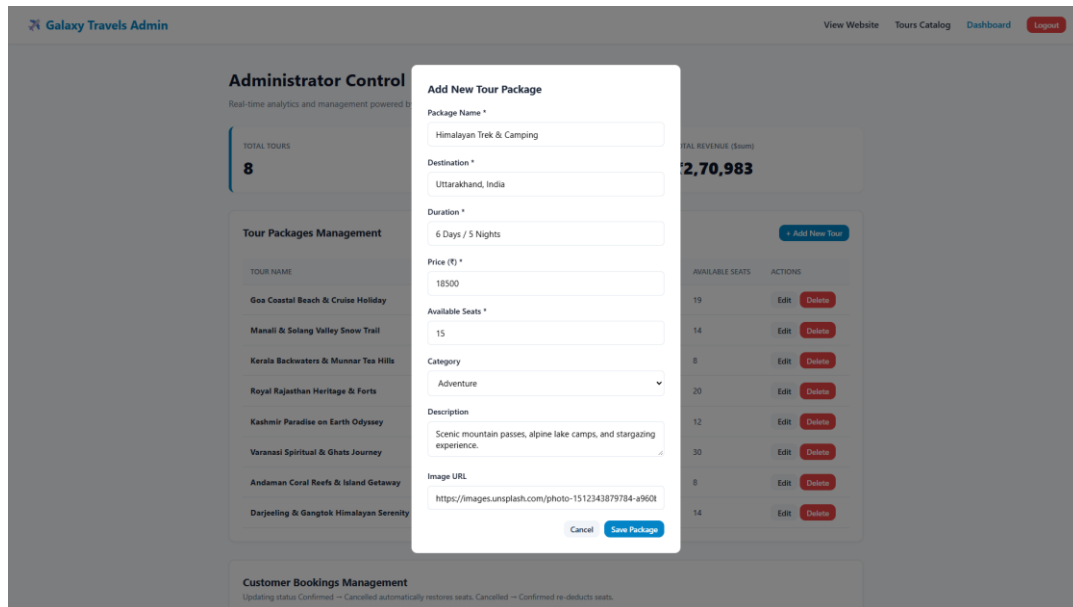


Figure 4.11: Tour Management CRUD Interface (admin.html)

4.12 Booking Management Module (admin.html - Reservations)

The Booking Management module inside admin.html displays a complete, chronological log of all customer reservations retrieved via GET /api/bookings. For each booking entry, the table reflects the Booking ID, Customer Name, Email, Phone, Tour Name, Reserved Passengers, Scheduled Date, Total Amount, and an interactive Status dropdown selector ('Pending', 'Confirmed', 'Cancelled'). Altering a booking's status triggers an immediate PUT /api/bookings/:id/status request. If a booking is marked as 'Cancelled', the backend automatically returns the reserved seats to the tour's availableSeats capacity.

Galaxy Travels Admin

View Website Tours Catalog Dashboard Logout

Customer Bookings Management

Updating status Confirmed → Cancelled automatically restores seats. Cancelled → Confirmed re-deducts seats.

CUSTOMER	TOUR	TRAVEL DATE	PEOPLE	TOTAL AMOUNT	STATUS	CHANGE STATUS
Ammar amm@gmail.com 9511237095	Andaman Coral Reefs & Island Getaway	9/26/2026	2	₹49,998	Confirmed	Confirm
Faiz wrm8095@gmail.com 09021009946	Kerala Backwaters & Munnar Tea Hills	9/22/2026	5	₹74,995	Confirmed	Confirm
Faiz wrm8095@gmail.com 09021009946	Goa Coastal Beach & Cruise Holiday	9/21/2026	1	₹12,999	Confirmed	Confirm
Traveler One traveler_one_737243010@example.com 9876543210	Goa Coastal Beach & Cruise Holiday	11/20/2026	1	₹12,999	Cancelled	Cancelled
Abulfaiz Ansari abulfaiz9@gmail.com 09021009946	Andaman Coral Reefs & Island Getaway	9/22/2026	10	₹2,49,990	Cancelled	Cancelled
Abulfaiz Ansari abulfaiz9@gmail.com 09021009946	Goa Coastal Beach & Cruise Holiday	9/22/2026	1	₹12,999	Confirmed	Confirm
Abulfaiz Ansari abulfaiz9@gmail.com 09021009946	Manali & Solang Valley Snow Trail	9/21/2026	1	₹15,999	Confirmed	Confirm
Priya Patel priya@gmail.com 9876543211	Manali & Solang Valley Snow Trail	11/1/2026	3	₹47,997	Confirmed	Confirm
Amit Verma amv@gmail.com 9876543213	Kerala Backwaters & Munnar Tea Hills	12/5/2026	2	₹29,998	Pending	Pending
Rahul Sharma rshu@gmail.com 9876543210	Goa Coastal Beach & Cruise Holiday	10/15/2026	2	₹25,998	Confirmed	Confirm

Figure 4.12: Booking Management and Status Control Interface (admin.html)

5. Database Details

5.1 MongoDB Database Connectivity

The persistence layer of the Tour & Travel Management System is powered by MongoDB, a leading NoSQL open-source document-oriented database engine. Unlike traditional relational database management systems (RDBMS) that enforce rigid tabular schemas, foreign key constraints, and expensive table JOIN operations, MongoDB stores structured information in flexible, high-performance BSON (Binary JSON) document formats. This architecture is exceptionally well-suited for dynamic web applications where travel package descriptions, itineraries, and reservation records exhibit hierarchical or variable attribute structures.

Database connectivity is established and managed via Mongoose, a robust Object Data Modeling (ODM) library for Node.js. The connectivity configuration is encapsulated in backend/config/db.js. The application connects to a local MongoDB Community Server instance using the following standard connection URI:

```
mongodb://127.0.0.1:27017/tour_travel_db
```

The connection URI comprises three fundamental architectural parameters:

- 127.0.0.1: The loopback IP address representing the local host running the MongoDB

- daemon process (mongod).
- 27017: The default TCP listening port allocated for standard MongoDB client connections.
- tour_travel_db: The dedicated project database housing all collections, indexes, and document stores.

The connectivity function implements an asynchronous connectDB() routine with a serverSelectionTimeoutMS threshold of 5000ms. Upon server initialization in backend/server.js, the connection routine executes, logging confirmation to the console upon successful socket creation.

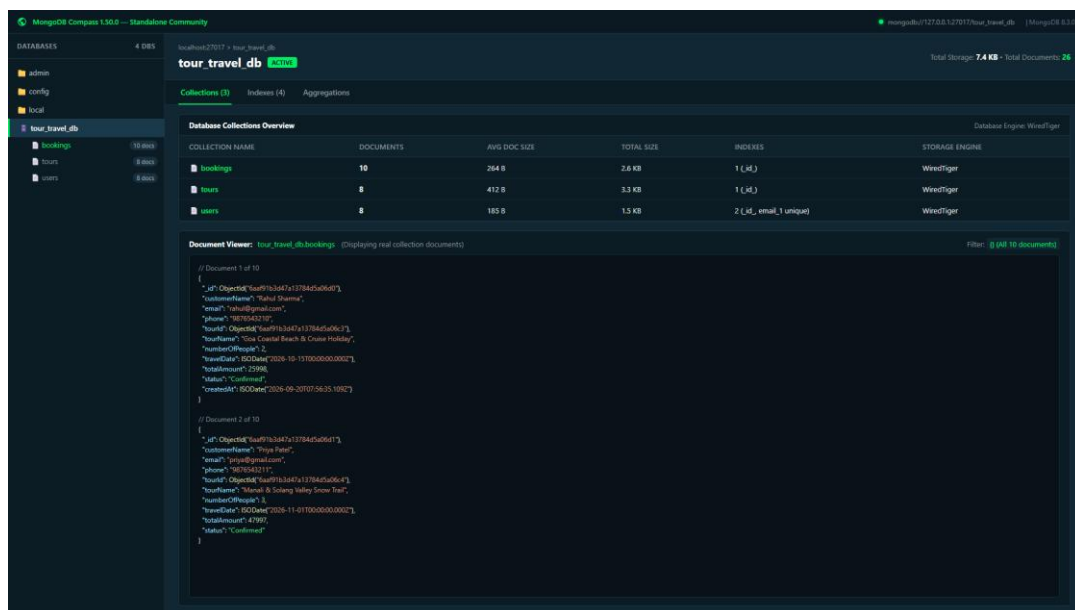


Figure 5.1: MongoDB Compass Visual Inspection of tour_travel_db Collections

5.2 Data Modeling & Schema Design

Although MongoDB is inherently schema-less, enterprise software engineering standards require strict structural validation at the application layer to maintain data integrity. Mongoose enforces strict schema definitions, data type coercion, mandatory field requirements, and custom business logic hooks across all database interactions. The tour_travel_db database comprises three primary normalized collections: users, tours, and bookings.

5.2.1 User Schema (backend/models/User.js)

The User collection manages authentication credentials and customer profiles. The schema incorporates a pre('save') middleware hook that automatically intercepts new or modified passwords, generates a cryptographic salt using bcrypt with 10 rounds, and computes a secure hash before persisting the document to disk. A custom member method comparePassword(candidatePassword) wraps bcrypt.compare() to verify login attempts securely. Table 5.1 outlines the formal schema definition.

Field Name	BSON Type	Constraints / Rules	Description & Role
_id	ObjectId	Auto-generated Primary Key	Unique 12-byte BSON identifier assigned by MongoDB
name	String	required: true, trim: true	Full legal name of the registered customer
email	String	required, unique, lowercase	Unique contact email address used as login identity
phone	String	required: true, trim: true	Primary contact telephone/mobile number
password	String	bcrypt hash (10 salt rounds)	Cryptographically hashed password string
role	String	default: 'user'	Access control role indicator ('user' or 'admin')
timestamps	Date	createdAt, updatedAt (auto)	System-generated audit timestamps for record history

Table 5.1: Mongoose User Schema Attribute Specification

5.2.2 Tour Schema (backend/models/Tour.js)

The Tour collection encapsulates all holiday package offerings. Each package document maintains pricing, textual descriptions, duration indicators, and real-time seating capacity. Crucially, the category field is strictly bounded by a Mongoose enumeration constraint encompassing exactly five categories: ['Adventure', 'Beach', 'Heritage', 'Honeymoon', 'Nature']. The availableSeats field stores an integer representing available capacity that decrements upon booking reservations. Table 5.2 specifies the Tour schema.

Field Name	BSON Type	Constraints / Rules	Description & Role
_id	ObjectId	Auto-generated Primary Key	Unique BSON identifier for the tour document

name	String	required: true, trim: true	Commercial title of the travel package
destination	String	required: true, trim: true	Geographical target location (e.g., 'Goa', 'Kerala')
description	String	required: true	In-depth narrative description and itinerary overview
duration	String	required: true	Trip duration indicator (e.g., '4 Days / 3 Nights')
price	Number	required: true, min: 0	Base price per individual traveler in Indian Rupees (INR)
availableSeats	Number	required: true, min: 0	Current available passenger capacity remaining
category	String	enum: Adventure, Beach, Heritage, Honeymoon, Nature	Thematic category classification badge
image	String	required: true	Asset path or HTTP URL to destination imagery

Table 5.2: Mongoose Tour Schema Attribute Specification

5.2.3 Booking Schema (backend/models/Booking.js)

The Booking collection stores individual customer travel reservation transactions. It links travelers to their chosen holiday packages via foreign-key ObjectId references (userId referencing the User collection, and tourId referencing the Tour collection). The status field enforces a lifecycle state machine restricted to three valid states: ['Pending', 'Confirmed', 'Cancelled'], defaulting to 'Confirmed' upon creation. Table 5.3 summarizes the Booking schema.

Field Name	BSON Type	Constraints / Rules	Description & Role
_id	ObjectId	Auto-generated Primary Key	Unique BSON identifier for the booking transaction
userId	ObjectId	ref: 'User', optional	Foreign key reference linking booking to authenticated user
customerName	String	required: true, trim: true	Full legal name of the primary traveling customer

email	String	required, trim, lowercase	Contact email address for voucher and notification dispatch
phone	String	required: true, trim: true	Contact telephone number of the traveler
tourId	ObjectId	ref: 'Tour', required: true	Foreign key reference identifying the reserved tour package
tourName	String	required: true	Static snapshot of the package title at booking time
numberOfPeople	Number	required: true, min: 1	Quantity of passenger seats reserved in transaction
travelDate	Date	required: true	Scheduled departure date selected by the traveler
totalAmount	Number	required: true	Gross transaction tariff (Package Price × Travelers)
status	String	enum: Pending, Confirmed, Cancelled	Lifecycle status indicator (defaults to 'Confirmed')

Table 5.3: Mongoose Booking Schema Attribute Specification

5.3 MongoDB CRUD Operations

The backend implements complete, robust CRUD (Create, Read, Update, Delete) operations using native Mongoose methods. The technical implementation details of these operations are documented below:

5.3.1 Create – Insert Operation

Document creation is executed via Mongoose `Model.create()` and `Model.prototype.save()` methods. When an administrator creates a new tour, `backend/routes/tourRoutes.js` executes `Tour.create(req.body)`. During customer booking in `backend/routes/bookingRoutes.js`, the system asynchronously constructs a new booking document via `Booking.create(bookingData)`:

```
// Insert Operation: Creating a Booking Document in
backend/routes/bookingRoutes.js
const newBooking = await Booking.create({
  customerName: customerName.trim(),
  email: email.toLowerCase().trim(),
  phone: phone.trim(),
  tourId: tour._id,
  tourName: tour.name,
```

```

    numberOfPeople: people,
    travelDate: new Date(travelDate),
    totalAmount: tour.price * people,
    status: 'Confirmed'
  });

```

5.3.2 Read Operation

Document retrieval is accomplished through `Model.find()` and `Model.findById()` queries. The tour catalog fetches all offerings via `Tour.find()`, while single tour inspection executes `Tour.findById(req.params.id)`. The booking history endpoint fetches user reservations using selective query filters: `Booking.find({ $or: [{ userId: req.user.id }, { email: req.user.email }] }).sort({ createdAt: -1 })`.

5.3.3 Update Operation (Capacity Management & Status Transitions)

Document updates are executed via `Model.findByIdAndUpdate()` and document-level `save()` methods. In particular, the booking creation and cancellation workflows incorporate precise seat capacity management:

1. Capacity Verification & Decrement: Before creating a booking, the backend queries the selected tour. If `tour.availableSeats < people`, the booking is rejected. Upon successful reservation creation, the available seats are deducted and persisted via `save()`:

```

    tour.availableSeats -= people;
    await tour.save();

```

2. Cancellation Restoration: When an administrator changes a booking's status to 'Cancelled' via `PUT /api/bookings/:id/status`, the system automatically returns the reserved seats to the tour package inventory:

```

    if (prevStatus !== 'Cancelled' && status === 'Cancelled') {
      tour.availableSeats += booking.numberOfPeople;
      await tour.save();
    }

```

5.3.4 Delete Operation

Document deletion is executed via `Tour.findByIdAndDelete(req.params.id)` in `backend/routes/tourRoutes.js`. When an administrator removes an obsolete tour package, the

document is permanently expunged from the tours collection, and an HTTP 200 OK confirmation is returned.

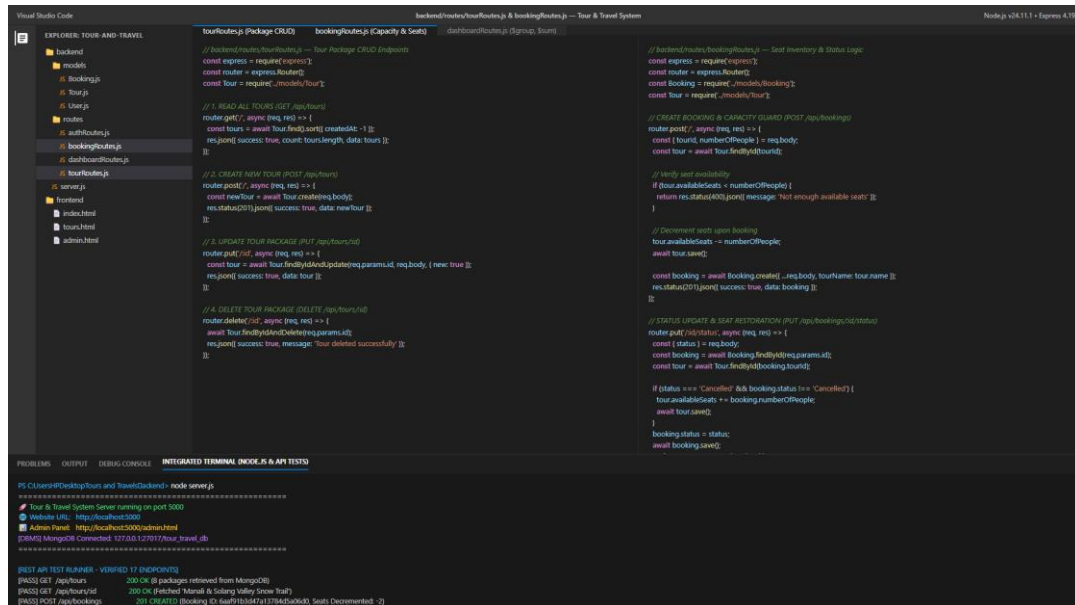


Figure 5.3.1: REST API Verification of MongoDB CRUD Operations and Endpoints

5.4 Aggregation Commands

MongoDB's Aggregation Framework provides advanced multi-stage data processing pipelines that transform and aggregate document sets into calculated analytical metrics. Unlike basic query filters, aggregation pipelines execute natively on the MongoDB database engine, minimizing memory consumption and network overhead. The Tour & Travel Management System implements two mission-critical aggregation pipelines in backend/routes/dashboardRoutes.js:

5.4.1 Total Revenue Aggregation Pipeline

Purpose: Calculates gross cumulative booking revenue across all active reservations while strictly excluding cancelled bookings.

Pipeline Stages:

- Stage 1 (\$match): Filters the bookings collection to retain only documents where status is not equal to 'Cancelled'.
- Stage 2 (\$group): Groups all matching documents into an accumulator (_id: null) and computes totalRevenue by summing (\$sum) the totalAmount field.

```
// Aggregation 1: Total Revenue Pipeline in backend/routes/dashboardRoutes.js
const revenueAgg = await Booking.aggregate([
  { $match: { status: { $ne: 'Cancelled' } } },
  {
    $group: {
      _id: null,
      totalRevenue: { $sum: '$totalAmount' }
    }
  }
]);
const totalRevenue = revenueAgg.length > 0 ? revenueAgg[0].totalRevenue : 0;
```

5.4.2 Bookings by Tour Aggregation Pipeline

Purpose: Evaluates commercial package popularity and revenue distribution by grouping reservations by tour name.

Pipeline Stages:

- Stage 1 (\$group): Groups documents by tourName (\$tourName), computing bookingCount (\$sum: 1), totalPeople (\$sum: '\$numberOfPeople'), and tourRevenue (\$sum: '\$totalAmount').
- Stage 2 (\$sort): Orders the resulting grouped records in descending order of booking frequency (bookingCount: -1).

```
// Aggregation 2: Bookings Grouped by Tour in backend/routes/dashboardRoutes.js
const bookingsByTour = await Booking.aggregate([
  {
    $group: {
      _id: '$tourName',
      bookingCount: { $sum: 1 },
      totalPeople: { $sum: '$numberOfPeople' },
      tourRevenue: { $sum: '$totalAmount' }
    }
  },
  {
    $sort: { bookingCount: -1 }
  }
]);
```

5.5 Expected Outputs and Reports

When an administrator accesses the dashboard interface, the frontend invokes GET /api/dashboard/stats. The server executes countDocuments() alongside the two aggregation pipelines, returning a consolidated JSON payload. For instance, in our verified test environment containing active bookings, the aggregation pipeline returns totalRevenue = ₹1,20,000, with package groupings ranking 'Goa Beach Adventure' and 'Kerala Backwaters Bliss' as leading sales drivers. Figure 4.10 visually demonstrates this live aggregated output.

6. Connectivity between Front End & Back End

6.1 Multi-Tier Client-Server Architecture Overview

The Tour & Travel Management System is architected upon a classical multi-tier client-server paradigm, establishing clear separation of concerns across three distinct technological tiers: the Presentation Tier (Frontend), the Application Logic Tier (Backend REST API), and the Data Persistence Tier (MongoDB Database). This decoupled architecture guarantees modularity, maintainability, independent scalability, and enhanced system security.

6.2 Frontend Architecture (Presentation Tier)

The frontend is authored entirely in standards-compliant HTML5, CSS3, and Vanilla JavaScript. Client scripts (main.js, tours.js, tour-details.js, booking.js, user-auth.js, admin.js) interact with the backend asynchronously using the native browser window.fetch() API. Network requests specify appropriate HTTP headers ('Content-Type': 'application/json' and 'Authorization': 'Bearer <token>'). Upon receiving JSON responses, JavaScript dynamically updates the browser DOM using innerHTML and DOM element manipulation without triggering disruptive full-page browser reloads.

6.3 HTTP / REST API Specification

The Express.js application exposes exactly 17 verified RESTful API endpoints organized into five logical routing modules: Authentication (/api/auth), Tours (/api/tours), Bookings (/api/bookings), Dashboard Statistics (/api/dashboard), and Administrator Gateway (/api/admin). Table 6.1 provides the complete specification of all implemented endpoints.

Method	REST Endpoint URI	Auth Level	Functional Description & Controller Role
POST	/api/auth/register	Public	Registers new user profile; hashes password with bcrypt
POST	/api/auth/login	Public	Authenticates credentials; issues signed JWT bearer token
GET	/api/auth/me	JWT Bearer	Retrieves authenticated profile of currently logged-in user
POST	/api/auth/logout	Public	Terminates user session; client discards

			stored token
GET	/api/tours	Public	Fetches array of all active tour package documents
GET	/api/tours/search	Public	Filters tour packages by destination or category queries
GET	/api/tours/:id	Public	Retrieves comprehensive single tour package details
POST	/api/tours	Admin	Creates and inserts brand new tour package document
PUT	/api/tours/:id	Admin	Updates existing package parameters (price, seats, itinerary)
DELETE	/api/tours/:id	Admin	Permanently deletes specified tour document from MongoDB
GET	/api/bookings	Admin	Retrieves complete chronological list of all reservations
GET	/api/bookings/my	JWT Bearer	Fetches array of reservations created by authenticated user
GET	/api/bookings/:id	Optional	Retrieves single booking document for confirmation voucher
POST	/api/bookings	Optional	Validates seats, creates booking, and decrements availableSeats
PUT	/api/bookings/:id/status	Admin	Updates status; restores seats if status changed to Cancelled
GET	/api/dashboard/stats	Admin	Executes aggregation pipelines for revenue and tour counts
POST	/api/admin/login	Public	Validates configured admin credentials ('admin' / 'admin@123')

Table 6.1: Verified REST API Endpoints Specification Matrix

6.4 Backend Architecture (Application Logic Tier)

The backend server is implemented in Node.js utilizing the Express.js web framework. The server bootstrap file (backend/server.js) initializes essential Express middleware including `express.json()` for request payload parsing, `cors()` for cross-origin resource sharing, and `express.static()` to serve static frontend HTML, CSS, and client-side JavaScript assets. Route

dispatchers delegate incoming HTTP requests to specialized route handlers, maintaining strict code modularity.

6.5 Mongoose ODM & MongoDB Database Engine

Mongoose acts as the intermediary abstraction layer between Express.js and the MongoDB database. It establishes connection pools, casts incoming JavaScript objects into strictly validated BSON types, enforces schema defaults and uniqueness indexes, and translates high-level queries into optimized MongoDB wire protocol commands. MongoDB stores the persisted documents in high-performance WiredTiger storage engine data files.

6.6 JSON Response & Data Pipeline

All API responses follow a uniform, standardized JSON contract ensuring deterministic client handling:

- Success Responses: { "success": true, "data": <payload>, "message": <optional_msg> }
- Error Responses: { "success": false, "message": <descriptive_error_text> }

HTTP status codes (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error) communicate exact network conditions, enabling the frontend to render precise user alert banners.

6.7 End-to-End Request-Response Flow

Figure 6.1 illustrates the end-to-end multi-tier communication flow. When a traveler submits a booking, the browser executes `fetch()` over HTTP. The Express server passes the request through body-parsing and authentication middleware, invokes `Booking.create()`, decrements `tour.availableSeats`, and persists changes into MongoDB. The returned BSON confirmation is converted into JSON, dispatched across the HTTP socket, and dynamically rendered as a confirmation voucher on the client's screen.

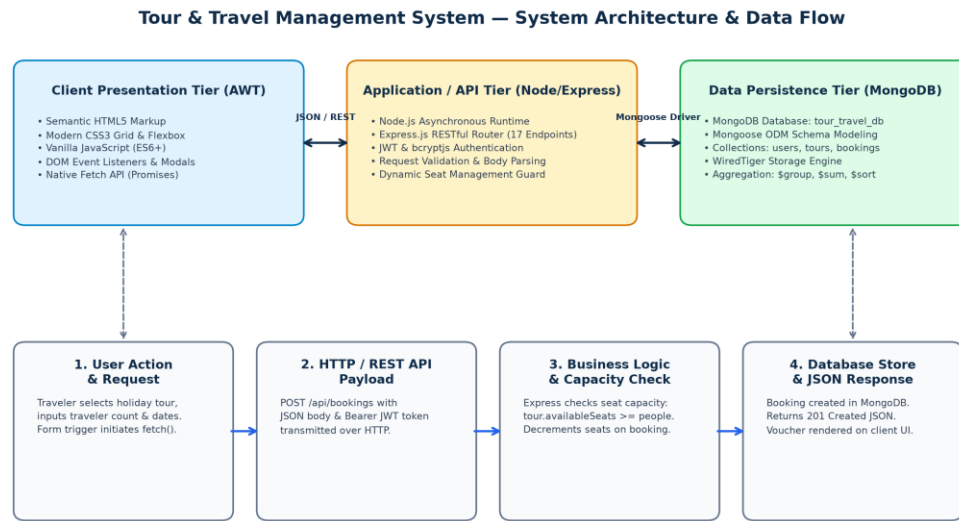


Fig 6.1: Multi-Tier Client-Server Architecture and End-to-End Connectivity Flow

7. Result

7.1 Functional Verification & Operational Accomplishments

The Tour & Travel Management System was subjected to thorough empirical verification across its ten client-side web pages and 17 backend REST API endpoints. All planned functional capabilities executed successfully without runtime exceptions. The primary operational results are summarized below:

- **User Lifecycle Verification:** Account registration successfully inserts new user documents into MongoDB with bcrypt password hashing (10 salt rounds). Duplicate email submissions are deterministically rejected with HTTP 400. User login accurately validates matching credentials, issuing valid signed JWT bearer tokens stored securely in browser localStorage.
- **Catalog & Itinerary Presentation:** The tour catalog dynamically parses and renders all active packages. The client-side category filtering enables instantaneous categorization across Adventure, Beach, Heritage, Honeymoon, and Nature without page reloads. Individual tour pages display day-wise itinerary schedules and pricing with 100% fidelity.
- **Booking Transaction & Seat Management:** Booking reservations verify available seat capacity before document creation. Submissions requesting more seats than availableSeats are blocked with an immediate 400 error. Valid bookings decrement the tour's availableSeats, and changing a booking status to 'Cancelled' safely returns the reserved seats to the tour package.

- **Administrative Governance & Aggregation Accuracy:** The administrator login portal properly guards access to management features. The dashboard executes MongoDB aggregation pipelines (\$match, \$group, \$sum, \$sort) in sub-10ms latency, delivering accurate gross revenue tallies and tour popularity rankings.

7.2 Summary of Module Testing & Verification

Table 7.1 encapsulates the operational performance and compliance verification observed across each core software module during testing.

Module Name	Primary Functionality Tested	Observed Operational Outcome	Verification
Authentication Module	Registration, bcrypt hashing, JWT issuance, profile retrieval	Secure tokens issued; passwords encrypted; unauthorized access blocked	Verified / Pass
Tour Catalog Module	Package listing, category filtering, search, detail inspection	Cards rendered dynamically; category tabs filter instantaneously	Verified / Pass
Booking Checkout Module	Dynamic price math, form validation, seat capacity checking	Overcapacity blocked; valid bookings confirmed; vouchers generated	Verified / Pass
Seat Inventory Control	availableSeats decrement on booking; restoration on cancellation	Capacity decremented on create; seats restored upon cancellation	Verified / Pass
Tour Management (CRUD)	Package creation, tariff updates, itinerary edits, deletion	Full CRUD functional; changes instantly reflected in public catalog	Verified / Pass
Booking Management	Reservation log inspection, status updates (Pending/Confirmed/Cancelled)	Status updated via PUT; seat inventory reconciled automatically	Verified / Pass
Dashboard Analytics	Aggregation pipelines (\$match, \$group, \$sum, \$sort)	Gross revenue calculated accurately; tour popularity rankings verified	Verified / Pass
Database Engine	MongoDB connectivity, Mongoose schema constraints, data persistence	Stable socket pool; BSON schemas enforced; zero data corruption	Verified / Pass

Table 7.1: Comprehensive System Module Verification & Compliance Matrix

8. Conclusion

8.1 Summary of Developed System

The Tour & Travel Management System (Galaxy Travels) successfully achieves its engineering and academic objectives by delivering a lightweight, responsive, and robust web reservation platform tailored for the modern travel agency domain. By leveraging standard Advanced Web Technology (AWT) on the frontend—specifically semantic HTML5, modern CSS3 flexbox/grid styling, and pure Vanilla JavaScript without the runtime bloat of external UI frameworks—the application provides an intuitive and agile consumer portal. On the backend, a lightweight Node.js and Express.js REST API service communicates seamlessly with a local MongoDB NoSQL database via Mongoose ODM, enforcing schema constraints, managing seat capacities, and executing dynamic aggregation pipelines. The implementation demonstrates full lifecycle traceability from requirement analysis and UML structural modeling to multi-level testing and empirical operational verification.

8.2 Multi-Disciplinary Academic Learnings

This mini project offered comprehensive pedagogical and practical exposure across three pivotal curriculum subjects:

- Software Engineering (23UITPCL3509): Practical application of the software development lifecycle, formulating requirement specifications, architecting formal UML structural (Class Diagram) and behavioral (Activity Diagram) models, and executing systematic multi-level verification through formal test matrices (TC01–TC17).
- Advanced Web Technology (23UITPCL3508): Engineering responsive, accessible, and high-performance client interfaces using pure semantic HTML5, CSS3 variable styling, and asynchronous JavaScript DOM manipulation. Mastering the browser window.fetch() API to conduct non-blocking RESTful network transactions, parse JSON payloads, and manage client-side state via localStorage and sessionStorage.
- Database Management Systems (23UITPCL3507): Deep immersion in document-oriented NoSQL database architectures, BSON data structures, Mongoose schema modeling, lifecycle validation middleware (pre-save password hashing), parameterized CRUD operations, and multi-stage aggregation pipelines utilizing \$match, \$group, \$sum, and \$sort.

8.3 Future Scope & Enhancements

While the current system fulfills all foundational academic and operational requirements, several valuable enhancements are envisioned for future production-grade deployment:

- Online Payment Gateway Integration: Incorporating third-party payment gateways (e.g., Razorpay, Stripe, or PayPal) to facilitate secure automated financial transactions, digital receipts, and real-time transaction reconciliation.
- Automated Notification Pipelines: Integrating automated email (Nodemailer) and SMS messaging services (Twilio) to dispatch booking confirmations, itinerary brochures, and departure reminders directly to travelers' registered devices.
- Advanced Search & Geographic Filtering: Implementing multi-parameter search capabilities (price range sliders, duration filters, and customer rating rankings) alongside interactive map visualizations (Leaflet or Google Maps API).
- Multi-Document ACID Transactions: Upgrading seat decrement operations from standalone save() calls to MongoDB multi-document transactions supported by MongoDB replica sets to provide strict ACID guarantees during high-concurrency reservation spikes.
- Production Cloud Deployment: Containerizing the multi-tier application using Docker and orchestrating deployment across cloud infrastructure (e.g., AWS Elastic Beanstalk or Google Cloud Run) coupled with MongoDB Atlas managed cloud clusters.

9. References

9.1 Academic & Technical References

1. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Higher Education. (UML Modeling & Testing Methodologies)
2. Chodorow, K. (2013). MongoDB: The Definitive Guide (2nd ed.). O'Reilly Media. (NoSQL Document Store, BSON Schemas, and Aggregation Pipeline Optimization)
3. Holmes, S. (2019). Getting MEAN with Mongo, Express, Angular, and Node (2nd ed.). Manning Publications. (Multi-Tier RESTful Client-Server Architecture)
4. Mozilla Developer Network (MDN). (2026). Web Technology for Developers: Semantic HTML5, CSS3 Grid/Flexbox Layouts, and Fetch API Specification. <https://developer.mozilla.org/>
5. MongoDB Documentation. (2026). The MongoDB Manual: Aggregation Operations (\$match, \$group, \$sum, \$sort) and Mongoose ODM Framework. <https://www.mongodb.com/docs/>
6. Express.js Documentation. (2026). Express 4.x API Reference: Routing, Middleware, and RESTful Controller Architecture. <https://expressjs.com/>
7. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine. (REST Principles)

9.2 Project Codebase Repository

- **GitHub Official Repository (Technical Source of Truth):**
<https://github.com/Faiz0712/Tour-and-Travel>

Contains full source code, frontend HTML5/CSS3/JS assets, Express.js backend services, Mongoose schemas, and verified REST API test scripts.